

TPO Programación 3 — Documentación técnica

Justificación de la técnica algorítmica y de los patrones de diseño reutilizados

Repositorio del proyecto: github.com/theoruschan/tpo-adivinanzas

1. Técnicas algorítmicas de cátedra: Greedy y Divide and Conquer

1.1 Qué pedía la cátedra

Los dos bloques de teoría vistos son Greedy (Huffman, Knapsack fraccionario, Cambio de monedas) y Divide and Conquer (Palíndromo, Fibonacci recursivo, pseudo-M). No es obligatorio usar las dos técnicas en el mismo trabajo: alcanza con identificar, dentro del problema propio del TPO, dónde aparece naturalmente alguna de ellas y aplicarla bien.

Este TPO terminó usando **las dos**, pero no como dos features separadas pegadas con cinta — aparecen en dos capas distintas del mismo problema (elegir qué preguntar):

- **Greedy** define el *criterio*: qué hace que un filtro sea "el mejor" en un turno (minimizar el peor caso posible, sin mirar turnos futuros). Ver 1.2 y 1.3.
- **Divide and Conquer** resuelve el *mecanismo*: cómo encontrar, entre los filtros disponibles, cuál cumple ese criterio (`MaquinaAvanzada.mejorPorDivideYVenceras`). Ver 1.3.

Y es igual de importante decir dónde **no** aparece D&C aunque a primera vista parezca que sí: en el filtrado de candidatos tras cada respuesta (`filtrarCandidatos`) no hay recursión ni combinación real, así que no se le fuerza esa etiqueta.

1.2 El problema real: elegir qué preguntar

`MaquinaAvanzada` (`jugador/MaquinaAvanzada.java`), en cada turno, tiene un conjunto de personajes candidatos y una lista de filtros todavía no usados. Son 4 características (género, calvicie, usa lentes, color de pelo), pero cada una se abre en variantes concretas — ¿es hombre?/¿es mujer?, ¿es calvo?/¿tiene pelo?, etc. — que en `FiltroFactory.todosFiltros()` (`modelo/FiltroFactory.java`) suman **9 filtros distintos** en total. Tiene que decidir qué filtro preguntar a continuación para achicar lo más posible ese conjunto, sin saber de antemano si la respuesta del rival va a ser "sí" o "no".

Es el mismo tipo de problema que Cambio de monedas o Knapsack fraccionario: en cada paso hay un conjunto de opciones disponibles, se evalúa cada una con un criterio numérico local, y se elige la mejor

sin mirar los pasos futuros. Nadie prueba todas las secuencias posibles de preguntas para las próximas rondas; se decide turno a turno con la información que hay en ese momento. Eso es, por definición, Greedy.

1.3 Cómo quedó implementado

`MaquinaAvanzada.elegirJugada(EstadoPartida estado):`

1. Si ya queda un solo candidato, no hay nada que evaluar: se arriesga la Suposicion directamente.
2. Si no, se delega en `elegirMejorFiltro(disponibles)` para decidir qué filtro preguntar.

La capa Greedy: el criterio. Lo que hace que un filtro sea "el mejor" está definido en `evaluar(Filtro) + EvaluacionFiltro.peorCaso()`: se llama a `JugadorMaquina.particionar(candidatos, filtro)`, que separa los candidatos actuales en dos grupos (cumplen / no cumplen), y como no se sabe qué va a responder el rival, se toma el peor de los dos casos — el grupo más grande. Es el criterio *minimax*: minimizar el máximo daño posible, decidido con la información de este turno únicamente, sin mirar turnos futuros ni volver atrás sobre una elección ya hecha. Es una elección greedy clásica, igual de "miope a propósito" que Cambio de Monedas o Knapsack fraccionario.

La capa Divide and Conquer: el mecanismo. Encontrar cuál de los filtros disponibles cumple ese criterio (el de menor `peorCaso()`) se resuelve con D&C, al estilo del ejercicio de cátedra de "mejor candidato" (encontrar el máximo/mínimo de un array dividiéndolo al medio):

```
private Filtro elegirMejorFiltro(List<Filtro> filtrosCandidatos) {
    EvaluacionFiltro mejor = mejorPorDivideYVenceras(filtrosCandidatos, 0, filtrosCandidatos.size() - 1);
    return mejor.getFiltro();
}

private EvaluacionFiltro mejorPorDivideYVenceras(List<Filtro> filtros, int lo, int hi) {
    if (lo == hi) {
        EvaluacionFiltro evaluacion = evaluar(filtros.get(lo));    // caso base
        ultimaEvaluacion.add(evaluacion);
        return evaluacion;
    }

    int mid = (lo + hi) / 2;
    EvaluacionFiltro mejorIzquierda = mejorPorDivideYVenceras(filtros, lo, mid);
    EvaluacionFiltro mejorDerecha = mejorPorDivideYVenceras(filtros, mid + 1, hi);

    // combina: el criterio minimax (menor peor-caso) decide cual mitad gana.
    return mejorIzquierda.peorCaso() <= mejorDerecha.peorCaso() ? mejorIzquierda : mejorDerecha;
}
```

Esto sí cumple la definición completa de D&C: se llama a sí mismo sobre subproblemas más chicos (mitades de la lista de filtros) y combina esos resultados en una respuesta final (comparando cuál mitad

trae el filtro con menor peorCaso()). A diferencia de filtrarCandidatos, acá **sí** hay una segunda llamada recursiva sobre cada mitad y **sí** hay un paso de combinación real.

Cómo se ve la recursión completa, con los 9 filtros reales. Vale la pena dibujarla, porque es fácil perder de vista en qué momento se calcula cada peor-caso y en qué momento se combina. Como 9 no es par, $mid = (lo + hi) / 2$ deja la partición despareja (5 filtros de un lado, 4 del otro) — pero el patrón es siempre el mismo en las dos mitades: dividir hasta que quede un solo filtro, recién ahí calcular su peor caso (evaluar), y combinar subiendo de a pares, nunca comparando un grupo de 2 o más sin terminar de dividirlo:

```
[9 filtros disponibles]
├─ lado A (5 filtros)
│   ├── [3 filtros]
│   │   ├── [2 filtros]
│   │   │   ├── filtro 1 → peor caso calculado
│   │   │   ├── filtro 2 → peor caso calculado
│   │   │   └─ comparo 1 vs 2 → gana uno
│   │   ├── filtro 3 → peor caso calculado
│   │   └─ comparo (ganador 1vs2) vs 3 → gana uno
│   ├── [2 filtros]
│   │   ├── filtro 4 → peor caso calculado
│   │   ├── filtro 5 → peor caso calculado
│   │   └─ comparo 4 vs 5 → gana uno
│   └─ comparo (ganador de arriba) vs (ganador 4vs5) → GANADOR DEL LADO A
├─ lado B (4 filtros)
│   ├── [2 filtros] → filtro 6 vs filtro 7 → gana uno
│   ├── [2 filtros] → filtro 8 vs filtro 9 → gana uno
│   └─ comparo esos dos ganadores → GANADOR DEL LADO B
└─ comparo GANADOR DEL LADO A vs GANADOR DEL LADO B
    └─ esa es la pregunta que se hace este turno
```

Ahí se ve clara la combinación de las dos técnicas de cátedra en una sola llamada: **Divide and Conquer decide el orden** en que se comparan las opciones (dividir, resolver cada mitad por separado, combinar); **Greedy decide el criterio** de esa comparación (quedarse siempre con el filtro de menor peor-caso, sin mirar turnos futuros ni la partida completa).

MaquinaBasica (jugador/MaquinaBasica.java) resuelve el mismo problema sin ninguna de las dos técnicas: elige un filtro al azar entre los disponibles. Esa es la comparación que pide el enunciado entre una máquina "menos acertiva" y otra más inteligente, y es lo que se puede observar tanto en el log de la partida como en el detalle de razonamiento que muestra la interfaz.

1.4 Complejidad de elegir un filtro

Con n candidatos actuales y k filtros disponibles ese turno ($k \leq 9$, la cantidad total de filtros concretos que hay en `FiltroFactory`, y baja de a uno por turno a medida que se van preguntando): evaluar un filtro (particionar) recorre los candidatos una vez, $\Theta(n)$.

`mejorPorDivideYVenceras` visita cada uno de los k filtros disponibles exactamente una vez (en las hojas de la recursión) y hace un trabajo $\Theta(1)$ de comparación por cada combinación — $T(k) = 2 \cdot T(k/2) + \Theta(1) = \Theta(k)$ llamadas en total, el mismo orden que recorrer la lista con un `for`. Encontrar el mínimo entre k opciones no ordenadas necesita inspeccionar las k una vez sí o sí, sea con un `loop` o con D&C: no hay forma de mejorar esa cota mirando menos opciones, porque cualquiera de las no miradas podría ser la mejor. El valor de usar D&C acá es estructural (demuestra la técnica de cátedra con recursión y combinación reales), no de performance.

Entonces el costo de un turno completo — evaluar los k filtros ($\Theta(k \cdot n)$, sin importar si la búsqueda del mínimo se organiza como `loop` o como D&C) más encontrar el mínimo entre ellos ($\Theta(k)$ adicional, insignificante al lado de $\Theta(k \cdot n)$) — sigue siendo $\Theta(k \cdot n)$. Como k es una constante chica (9 filtros fijos, nunca más), el costo por turno es en la práctica $\Theta(n)$, igual de barato que ordenar por valor/peso en `Knapsack` o recorrer las monedas en `Cambio`. Ver 1.5 para la cota del costo total de toda la partida (no solo de un turno).

1.5 Complejidad total de la partida y por qué no conviene ir más allá con D&C

El D&C que se agregó en 1.3 (`mejorPorDivideYVenceras`) es chico a propósito: solo decide entre los k filtros *ya evaluados* de **este** turno, nunca mira turnos futuros ni explora la rama que el rival no respondió. Antes de seguir, vale la pena distinguirlo del D&C "grande" del que habla esta sección — uno hipotético que construiría el árbol de decisión completo de toda la partida — para que no se confundan: uno se usa (1.3), el otro se evaluó y se descartó, y son dos escalas completamente distintas del mismo problema.

Costo del Greedy actual. Dentro de un mismo turno, `elegirMejorFiltro` (1.3) recorre la lista completa de candidatos **una vez por cada filtro disponible ese turno** — no hay forma de evitarlo, sea con un `loop` o con D&C (1.4): para saber cuál filtro conviene hay que particionar con todos y comparar, no solo con uno. Con k_t filtros disponibles en el turno t y n_t candidatos en ese momento, el costo de ese turno es $\Theta(k_t \cdot n_t)$.

Lo que mantiene esto barato es que ni k_t ni la cantidad de turnos dependen de n : hay 9 filtros en total (`FiltroFactory`) y se gasta uno por turno, así que k_t arranca en 9 y baja de a uno (9, 8, 7, ..., 1), y la partida nunca dura más de 9 turnos preguntando — al agotar los filtros, la máquina arriesga (`MaquinaAvanzada.java:41-44`). Sumando el costo de todos los turnos:

$$\sum (k_t \cdot n_t) \quad \text{para } t = 1..9, \quad \text{con } k_t = 10-t \quad \text{y} \quad n_t \leq n_0 \quad (\text{el catálogo inicial})$$

$$\leq n_0 \cdot (9 + 8 + 7 + \dots + 1) = n_0 \cdot 45$$

45 es una constante fija — no crece si el catálogo tiene 20 personajes o 20.000 — así que el costo total de decidir toda la partida sigue siendo $\Theta(n)$: un múltiplo constante de una sola pasada sobre el catálogo, nunca $\Theta(n^2)$ ni algo que dependa de cuántos turnos tenga el juego. En la práctica es bastante más barato todavía que esa cota, porque n_t baja fuerte turno a turno (el criterio minimax elige justamente el filtro que más achica el peor caso), no se queda estancado cerca de n_0 .

Costo de un D&C "de árbol completo" sobre este problema. Para eso, en cada nodo de decisión habría que recursar sobre las dos ramas (cumplen y no cumplen) y combinar los resultados para elegir la secuencia de preguntas que minimice el total de turnos de toda la partida, no solo el siguiente paso — es decir, construir el árbol de decisión óptimo. Eso implica:

- No se puede descartar la rama que "no ocurrió" como hace hoy `filtrarCandidatos`: hay que explorar y resolver **ambas** ramas para poder comparar combinaciones, porque no se sabe de antemano cuál va a pasar en la partida real.
- Con k filtros por explorar en cada nivel, el espacio de árboles a comparar crece combinatoriamente (del orden de $k!$ secuencias posibles, y en cada una se vuelve a tocar los candidatos en cada nivel del árbol). Con los 9 filtros reales de este dominio, $9! = 362.880$ secuencias posibles — ya no es un número chico ni siquiera con k fijo en 9. Es la misma razón por la que construir un árbol de decisión óptimo es, en general, un problema NP-difícil: no escala si se agregan más filtros, y ya pesa incluso con la cantidad fija que hay hoy.
- La mitad de ese trabajo, además, se termina descartando: en la partida real solo se recorre la rama que efectivamente responde el rival; la otra rama, explorada durante la construcción del árbol "óptimo", nunca se llega a usar.

	Greedy + D&C acotado (lo implementado, 1.3)	D&C de árbol completo (hipotético)
Qué mira	Solo el turno actual	Todas las secuencias futuras posibles
Complejidad	$\Theta(n)$ total para toda la partida	Combinatoria en la cantidad de filtros (del orden de $k!$)
Garantía	Ninguna de mínimo global	Sí, la secuencia de preguntas óptima
Por qué cuesta más	—	Tiene que explorar y combinar ambas ramas en cada nodo para poder comparar árboles completos

La razón por la que Greedy gana en complejidad acá no es casualidad: es el trade-off clásico velocidad-contra-optimalidad. El D&C sería más lento *porque* resuelve un problema más difícil

(garantizar el óptimo global explorando ambas ramas en cada nodo), mientras que Greedy se conforma con la mejor decisión local y por eso puede resolverse con una sola pasada. No es solo que este D&C "de árbol completo" no encaje estructuralmente en el problema de reducir candidatos: si se lo forzara a encajar ahí, además saldría peor en complejidad, no mejor. Esa es la diferencia con el D&C que sí se usa (1.3): ese resuelve un subproblema mucho más chico y acotado (elegir el mínimo entre k valores ya conocidos de este turno), no el problema completo de construir la partida óptima.

2. Patrones y separación aplicados en el diseño

Además de la técnica de cátedra, el TPO reutiliza algunas ideas de diseño estándar para mantener separadas las responsabilidades:

Strategy en ModoJuego. La interfaz `ModoJuego` (`controlador/ModoJuego.java`) tiene tres implementaciones intercambiables — `JugadorVsMaquina`, `MaquinaVsMaquina`, `JugadorVsDosMaquinas` —, cada una con su propia forma de armar los jugadores y de decidir el objetivo de cada turno. `MotorDeJuego` y `Partida` reciben un `ModoJuego` por parámetro y nunca preguntan de qué implementación concreta se trata: así se pueden agregar modos de juego nuevos sin tocar el motor.

Jerarquía Jugador → `JugadorMaquina` → `MaquinaBasica` / `MaquinaAvanzada`. La clase abstracta `Jugador` concentra lo común a cualquier jugador (encapsular el personaje secreto, responder filtros sin exponerlo). `JugadorMaquina` agrega lo común a cualquier máquina (la lista de candidatos, particionar, `filtrarCandidatos`). Lo único que varía entre `MaquinaBasica` y `MaquinaAvanzada` es la implementación de `elegirJugada`: una elige al azar, la otra con el criterio greedy. Esa es justamente la pieza que hace falta para comparar una estrategia simple contra una optimizada.

Vista intercambiable. La interfaz `Vista` (`vista/Vista.java`) define todo lo que el motor necesita mostrar y pedir, sin saber nunca si del otro lado hay una consola o una ventana. `VistaConsola` y `VistaSwing` son dos implementaciones de esa misma interfaz. Gracias a esta separación, agregar la interfaz gráfica no requirió tocar ni una línea de `MotorDeJuego`, `MaquinaBasica`, `MaquinaAvanzada` ni de los `ModoJuego`: toda la lógica de juego quedó intacta, solo cambió qué implementación de `Vista` se le pasa al `ControladorPrincipal` al arrancar (`app/Main.java`).

Persistencia aislada. `MarcadorRepositorio` es la única clase que conoce el formato del archivo de marcador. El resto del sistema solo le pide "sumar una victoria a este usuario" o "dame el top N"; si el día de mañana se cambia texto plano por una base de datos, ese cambio queda contenido en una sola clase.

3. La interfaz gráfica (Swing)

VistaSwing (vista/VistaSwing.java) es una única ventana, sin diálogos modales tapando nada:

- El log grande de la izquierda funciona como la "consola" del juego: ahí se ve todo el detalle de cada turno, incluido el razonamiento greedy completo de MaquinaAvanzada (qué filtros evaluó, cuántos candidatos le quedarían con cada uno, cuál eligió y por qué) — el mismo nivel de detalle que antes solo se veía por consola real.
- El panel fijo de la derecha muestra, en todo momento, el personaje secreto propio (con todas sus características) apenas se elige, y la lista de candidatos que le van quedando al jugador humano. Ninguno de los dos requiere abrir una pantalla aparte para consultarlos.
- Todo lo que hay que responder (elegir del menú, elegir personaje, elegir filtro, escribir el nombre de usuario, confirmar "continuar") se resuelve con los controles fijos de abajo (un combo o un campo de texto más un botón), nunca con una ventana emergente que bloquee el resto de la pantalla.

Como MotorDeJuego corre bloqueando (espera cada respuesta antes de seguir), y ese bloqueo no puede pasar en el hilo de eventos de Swing (la EDT) sin trabar toda la ventana, el motor corre en un hilo aparte (app/Main.java). La comunicación entre ese hilo y la ventana se resuelve con una cola de un solo lugar (BlockingQueue): el hilo del juego arma el control correspondiente y se queda esperando en la cola; el botón, que corre en la EDT, deja la respuesta ahí apenas el usuario confirma. Así la ventana nunca deja de responder ni de poder scrollarse mientras se espera una decisión.